

Auction, but Don't Block

Xiaowei Yang

Department of Computer Science
University of California, Irvine
xwy@uci.edu

ABSTRACT

This paper argues that ISP's recent efforts to block certain applications and attempts to differentiate traffic might be a signal of bandwidth scarcity. Bandwidth-intensive applications such as VoD might have driven the traffic demand to the capacity limit of their networks. This paper proposes to let ISPs auction their bandwidth, instead of blocking or degrading applications. A user places a bid in a packet header based on how much he values the communication. When congestion occurs, ISPs allocate bandwidth to those that value it the most, and charge them the Vickrey auction price. We outline a design that addresses the technical challenges to support this auction and analyze its feasibility.

1. INTRODUCTION

The success of the Internet is largely due to its openness [7]. Openness invites innovation. New applications can attach to the Internet without the permission of the network, and users can access any content or application at will. It is the myriad Internet applications that generate new value, improve efficiency, and advanced the societal life into an information era.

However, a recent trend raises serious concerns about the future of the Internet [17]. The CEO of a large ISP publicly announced the intention not to give Internet upstarts (e.g. Google, Yahoo!) a "free-ride" [4], and novel applications such as VoIP are blocked or degraded [2,5]. Were this trend to continue, the Internet might gradually erode into a closed network and future innovations might be stifled.

An earlier proposal [18] calls for using techniques such as data encryption and IP anonymization to fight the trend. This approach will inevitably escalate the tussle between users and ISPs: ISPs will use more sophisticated tools such as traffic analysis to block or discriminate, and users will then deploy counter techniques such as stenography to evade those tools, and ISPs will in turn try to crack those techniques, and so on.

This paper tackles this problem from a different perspective. We give ISPs the benefit of doubt that their actions to block or discriminate are justifiable, as clarified in [10,13]: bandwidth-intensive applications have created a bandwidth demand that reaches the capacity limit of their networks. Continuously upgrading their

networks is too costly. Hence, when there is not enough bandwidth for everyone, it is reasonable to differentiate.

This paper proposes to let ISPs auction their bandwidth among users, instead of blocking or degrading applications without considering their values to users. A user explicitly signals how much he values a network communication by placing a bid in a packet header. ISPs prioritize packets according to their bids if congestion occurs. To prevent a user from overstating his valuation or probing the network for the minimum winning bid, ISPs charge the user the highest bid of a packet that does not receive a prioritized service. Conceptually, this is a Vickrey auction [11,12]. Users have incentives to bid their true valuations.

This proposal has a number of advantages. First, it helps to keep the net open. It is the users that decide which packets receive prioritized services. An ISP that merely intends to address the problem of bandwidth scarcity (not exporting their market power to the application layer) does not have to decide which applications to block or degrade. If congestion occurs, applications that have low values to users are automatically slowed down, and network resources are efficiently allocated to those that value them the most. Second, ISPs do not need to be the "evil-doers" that raise prices to provide quality of service. Instead, it is the users that compete among themselves for the limited capacity ISPs currently have. Third, it is possible that ISPs can absorb some consumer surplus from highly valued applications without content or application based differentiation. For instance, if a user wants to watch an online movie on a Saturday night but the network is congested, he may be willing to pay a few extra dollars to the network to receive a high-quality video stream.

Packet auction was previously proposed by Mackie-Mason and Varian [12], but lacks a companion technical design. The primary contribution of this work is a design that enables packet auction. The design addresses a number of challenges, including how to efficiently and robustly support auctions at multiple providers, how to do practical billing and accounting, how to support sender-receiver joint bid, and how to handle packet losses. The design does not require per-packet micro-payment, and fits into the present bilateral billing model of ISPs.

A preliminary feasibility analysis suggests that the design is computationally efficient and has moderate header overhead.

The rest of this paper is organized as follows. We discuss related work in § 2. § 3 describes the design and § 4 provides a feasibility analysis. We discuss how end systems may place bids on packets in § 5. We conclude in § 6.

2. RELATED WORK

Mackie-Mason and Varian [12] propose to use packet auction (the *smart market*) to account for the congestion costs that one packet causes on others, but do not provide a technical design. Semret [14] analyzes different bidding strategies and their convergence properties.

Other researchers propose to use explicit congestion prices, rather than auction, to reduce traffic demand when congestion occurs. Gibbens and Kelly [9] propose to mark flows proportional to their sending rates when congestion occurs, and charge marked packets at congestion prices. Re-ECN [3] gives senders the incentive to echo congestion markings so that they can be charged directly by their access networks. Edge pricing and expected capacity pricing [6, 15] propose to charge a user based on a capacity profile, and shape his traffic according to the profile.

Explicit congestion pricing has the drawback that the congestion price is difficult to set [12, 15]. This is because congestion is dynamic, and the consequence of congestion, delayed transmission, has different costs on different users and applications. This work aims to provide a technical design that enables packet auction. It complements existing work on congestion pricing, and provides the market an additional option to address the problem of resource scarcity.

Bill-pay [8] is a system that allows users to place a micro-payment in a packet header to choose the desired quality of service. ISPs bill each other at the end of a billing cycle based on the micro-payment they take from each other's traffic. In our design, users place bids, not explicit payments. In addition, we constrain the billing scheme to fit within the present bilateral billing model, in which adjacent ISPs pre-negotiate who-pays-whom when they interconnect.

3. DESIGN

This section sketches a design that supports packet auction. The key challenges we address include: 1) how to implement the auction efficiently, 2) how to bill users, 3) how to do robust and efficient accounting, and 4) how to handle packet losses.

3.1 Packet Auction With Asynchronous Arrivals

A packet auction does not exactly map to a real-life auction because packets arrive asynchronously. Pack-

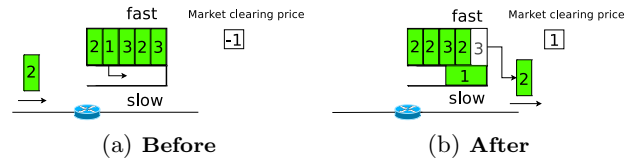


Figure 1: The market clearing price is changed from -1 to 1 when a packet with the lowest bid 1 is demoted from the fast lane. A packet leaves from the fast lane is charged at the price 1.

ets with different bid values may be discarded before a packet enters or leaves the queue, yet a queue may not be full when the packet leaves or arrives. At which value should the packet be charged? Moreover, how should a router handle a packet that loses its bid? If the router discards the packet immediately, flows with losing bids may be starved.

To address those issues, our design uses two queues to implement packet auction. A router maintains a fast lane and a slow lane by assigning different weights to the queues, e.g., assigning 80% of its bandwidth to the fast lane, and 20% to the slow lane. These two queues are work-conserving.

A router uses the bid in a packet header to determine which packets enter the fast lane, and a packet with a losing bid is enqueued at the slow lane to prevent starvation. For simplicity, we assume all packets are of unit size at the moment. We discuss the issue of various packet sizes in § 3.6. The highest bid among packets in the slow lane is referred to as the market clearing price.

When a packet arrives, if its bid is not above the market clearing price, it is enqueued into the slow lane. Otherwise, it is enqueued into the fast lane. If the fast lane is full, then the bid in this packet is compared against all packets in the fast lane. A packet with the lowest bid is demoted to the slow lane, and the market clearing price is updated to the bid of the demoted packet. If there are multiple packets with the same lowest bid, either randomness or FIFO order can be used to break the tie.

It is possible that the slow lane is full when a packet is demoted. If this demoted packet is discarded, then the market clearing price is inaccurate. In this case, a random packet with a bid lower than the market clearing price is selected and discarded. Our design does not further prioritize packets in the slow lane based on their bids in order to provide flows with losing bids a roughly equal chance to receive a demoted packet. It is also possible that the demoted packet arrives earlier than some packets in the slow lane. To avoid further penalizing a demotion, our design inserts a demoted packet into the slow lane according to its arrival time, i.e., the FIFO order.

A fast lane packet will be charged at the market clearing price (if it is non-negative) when it leaves. The bid

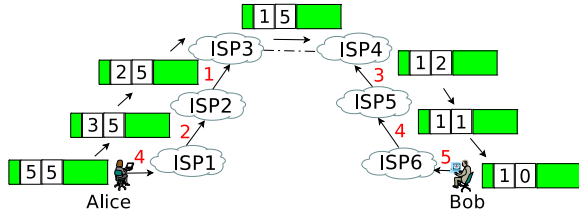


Figure 2: A packet carries a sender bid 5 (left) and a receiver bid 5 (right). A provider bills a customer the sum of the bidding costs incurred at its network and all upper-level providers' networks.

in its header is decreased by the amount of the market clearing price. This process simulates a Vickrey auction among all packets that have arrived in the queues when a fast lane packet departs: the packet pays the highest bid that loses the fast lane service at that time. A packet that departs from the slow lane is not charged. But after it departs, the market clearing price is updated to the highest bid among the remaining packets. When the slow lane becomes empty, we set the market clearing price to -1. This allows a packet with a bid zero to enter the fast lane when the slow lane is empty.

When a packet is demoted, our design sets a demotion bit to notify the sender. This notification is similar to ECN. The sender is expected to take actions to avoid losing its packets in the future. Packets without a bid are treated as if they have a zero bid, receiving a default best effort service.

Figure 1 shows an example. When a packet with a bid 2 arrives, the fast lane is full, and the market clearing price is -1. The packet with the lowest bid 1 in the fast lane is demoted to the slow lane, and the market clearing price is increased to 1. When a packet with a bid 3 departs from the fast lane, it is charged at the price 1, and its bid is decreased to 2.

A packet may traverse multiple links. At each link, the packet will be charged at the market clearing price of that link if it gets into the fast lane. Our design ensures that any packet in the fast lane has a bid no less than the market clearing price. Thus, a packet will not be left with a negative bid.

3.2 The Billing Model

We describe how a provider may bill for the bidding cost of a customer's traffic. We constrain our design to use the existing Internet billing model to be practical. The bidding costs can be added as an additional item in a customer's bill besides the existing costs such as usage-based fees or connection fees.

To support this billing scheme, our design separates a bid in a packet into a sender bid b_s and a receiver bid b_d . A bidding header in a packet thus includes a sender portion and a receiver portion. The sender bid b_s is used to enter auctions in the sender's providers'

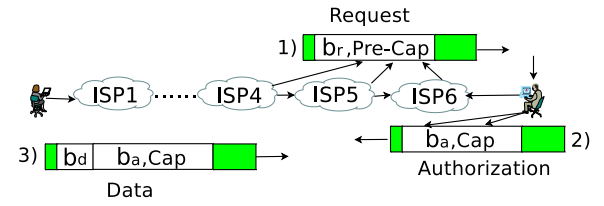


Figure 3: A sender and a receiver negotiate a verifiable bid using extended network capabilities. Only the receiver's bid header is shown for clarity.

networks, and b_d is used for auctions in the receiver's providers' networks. An ISP charges a customer the bidding costs incurred by the customer's traffic at itself and all its upper-level providers.

Figure 2 shows an example of how ISPs bill their customers. ISP_3 and ISP_4 are two tier-1 providers. An arrowed line points from a customer to a provider, and the digit next to a line is the amount of charge. Suppose Alice sends a packet with a sender bid 5 and a receiver bid 5 to Bob, and it is charged a total of four units of payment in Alice's providers' networks, and five units of payment in Bob's providers' networks. ISP_1 charges the sender Alice the sum of the costs incurred at all upper-level providers and itself: ISP_1 , ISP_2 and ISP_3 , a total of four. ISP_2 charges ISP_1 two units of payment for the bidding costs of ISP_1 's traffic within its network and its provider ISP_3 's network. Similarly, ISP_3 charges ISP_2 one unit of payment for the bidding cost of ISP_2 's traffic. A similar process happens on the receiver side.

3.3 Receiver Authorization

The above billing model raises the question how a receiver's provider verifies that a receiver has authorized a bid b_d . This issue can be handled by extending network capabilities [19]. We briefly describe this process in Figure 3, and more details on capabilities can be found in [19]. The sender bid header is omitted for clarity. A sender first sends a request packet to request a receiver bid level b_r . ISPs stamp pre-capabilities. If its policy allows, the receiver returns an authorized bid b_a and capabilities to the sender. The sender can then send packets with a receiver bid b_d and capabilities. The capabilities specify the authorized bid b_a . A receiver's provider honors b_d if the capabilities verify and the receiver's bid b_d is no more than the authorized bid b_a . The first receiver's provider on the path must also verify that the original b_d equals b_a for accounting purpose (§ 3.4). As an optimization, b_a and capabilities can be cached by a receiver's providers to reduce packet header overhead [19].

3.4 Accounting

Next, we discuss how an ISP accounts for the bid-

ding costs of a customer. The challenge is how an ISP obtains the costs incurred by individual customers at upper-level providers.

3.4.1 The Receiver Side

For a customer's inbound traffic, a provider may account for the bidding costs by summing up the difference between the authorized receiver bid b_a and the current receiver bid b_d of a customer's traffic. This difference is the total charge of a packet by all upper-level providers and the provider itself. The provider can keep a counter C_u for each customer u . When a packet is forwarded to u , it increases C_u by the amount $(b_a - b_d)$. At the end of a billing cycle, the provider bills the customer u the amount C_u .

3.4.2 The Sender Side

The accounting problem at the sender side (i.e. a customer's outbound traffic) is complicated because the upper-level providers are downstream on the path. A sender's provider does not know the bidding costs at those providers when it receives a packet. One possibility is to ask an upper-level provider to inform its customers the bidding charges of their traffic. However, an upper-level provider must break down the charges to the granularity of individual subscribers. Otherwise, an access provider is unable to accurately bill a subscriber for the bidding cost of his traffic. This requirement may become a scalability issue at large providers.

Our design trades accuracy for scalability. It enables an access ISP to charge individual subscribers accurately for the bidding costs of their traffic, but lets ISPs bill large customers, e.g. other ISPs or organizations, based on statistical averaging. We think that large providers would have sufficient traffic aggregation to ignore cost variations over time. We describe how to obtain the average bidding cost and an individual subscriber's bidding cost respectively.

Obtain Average Bidding Costs: The high-level idea is to let an ISP obtain the average market clearing prices from its providers, and charge a customer at the average prices from the bids it has observed. We describe this process formally using a fluid model. Let $M_l(t)$ denote the market clearing price of a provider l at time t , and $\wp_h = lP_1P_2\dots P_h$ denote a path that starts at the provider l , ends at a provider P_h , and includes only customer-provider links. Let P_0 denote the provider l in the path $\wp_h$, $T_{u,\wp_h}^l(t)$ denote the rate of the traffic from a customer u to a provider l that is forwarded along the path $\wp_h$, and $B_{u,\wp_h}^l(t)$ be the distribution of bids of this traffic. The bidding costs $\Delta C_u^l(t)$ incurred by u 's traffic at l and l 's providers in an interval $[t, t + \Delta t)$ can be computed as follows:

1. **for** each path $\wp_h$: **do**
2. **for** $i = 0, \dots, h$: **do**

3. **if** $(B_{u,\wp_h}^l(t) \geq M_{P_i}(t))$ **then**
4. $\Delta C_u^l(t) += T_{u,\wp_h}^l(t) \cdot M_{P_i}(t) \cdot \Delta t$
5. $B_{u,\wp_h}^l(t) -= M_{P_i}(t)$

Line 3-5 simulate how the amount of traffic $T_{u,\wp_h}^l(t) \cdot \Delta t$ with the bid $B_{u,\wp_h}^l(t)$ is charged at each provider P_i along the path $\wp_h$. The provider l can obtain the total bidding cost of a customer u by integrating $\Delta C_u^l(t)$ over a billing cycle: $\int \Delta C_u^l(t)$.

Our design uses the average market clearing price $\overline{M_j(t)}$ to approximate $M_j(t)$, as a provider l does not know the instantaneous value $M_j(t)$ of a downstream provider j . A provider l keeps a histogram of u 's traffic binned by its bids and paths over an interval Δt . Let $\Delta T_{u,\wp_h,b}^l(t)$ be the amount of traffic in a bin of the bid value b . Each provider j sends a customer its average market clearing price $\overline{M_j(t)}$ averaged over Δt , and the customer propagates this value to its customers and so on. The provider l uses the traffic histogram and the average market clearing prices from its upper-level providers to approximate the computations in Line 3-5 as follows:

6. **for** each $b \geq \overline{M_{P_i}(t)}$ in the ascending order: **do**
7. $\Delta C_u^l(t) += \Delta T_{u,\wp_h,b}^l(t) \cdot \overline{M_{P_i}(t)}$
8. $\Delta T_{u,\wp_h,(b-\overline{M_{P_i}(t)})}^l += \Delta T_{u,\wp_h,b}^l(t)$
9. $\Delta T_{u,\wp_h,b}^l(t) = 0$

Line 8-9 simulate the case that after the amount of traffic $\Delta T_{u,\wp_h,b}^l$ with a bid b is charged the average market clearing price $\overline{M_{P_i}(t)}$, its bid becomes $b - \overline{M_{P_i}(t)}$.

The averaging interval Δt can be short for a higher accuracy, or as long as a billing cycle for simplicity. We think that it can be set to five minutes to be consistent with the commonly used 95% - 5 billing scheme.

Obtain a Subscriber's Bidding Cost: If a sender sends a packet with a bid b_s , and when the packet exits the sender's last provider on the path, the bid becomes b'_s , then the bidding cost for this packet is $b_s - b'_s$. The sum $\sum b_s - \sum b'_s$ over all traffic of the sender is the total bidding cost of the sender. An access provider can easily track the sum $\sum b_s$ when it receives a sender's packets. The challenge is to obtain the exit bid b'_s .

Our design lets a receiver echo back the exit bid b'_s to the sender (e.g. piggyback on TCP ACKs), and the sender submit this value to its access provider on a subsequent packet. If a sender does not return a bid b'_s , it may be charged more than its actual cost.

Our design uses a keyed one-way hash function to prevent a sender from lying about the exit bid b'_s , as shown in Figure 4. When an access provider receives a packet from a sender, it inserts a nonce n and a key k into the packet header, as shown in step 1 in Figure 4. The key k can be generated from a time-varying master key, the nonce n , and the sender's IP address, using a

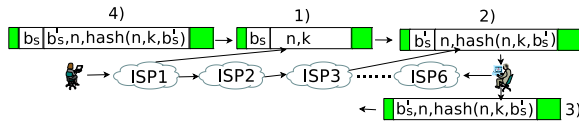


Figure 4: An access provider uses a keyed one-way hash function to prevent a sender from modifying its exit bid. Only the sender bid header is shown for clarity.

pseudo random number generator or a hash function. When the packet exits the sender's last provider on the path, e.g., ISP_3 in Figure 4, the exit provider computes a keyed hash over the exit bid b'_s : $hash(n, k, b'_s)$, and replaces the key with the hash value. The receiver echos back b'_s , the nonce n , and the hash value as shown in step 3 in Figure 4. The sender submits the exit bid b'_s , the nonce n , and the hash value in a subsequent packet (step 4 in Figure 4). The access provider recovers the key k from its master key, the nonce n , and the sender's IP, and verifies $hash(n, k, b'_s)$. If a sender lies about the exit bid b'_s , the hash value would not verify. The nonce n prevents a sender from submitting the same exit bid twice. The access provider changes its master key every few seconds to prevent a sender from guessing the key k using brute force search. If a sender does not submit its exit bid within the key expiration time, the hash value will not verify and a sender will be charged more.

3.5 Packet Losses

Packet loss impacts the accuracy of accounting. If a packet is discarded in the middle of the network, the receiver can not echo back the exit bid to the sender, and the sender will be charged more. One possibility to improve the accuracy is to use the packet obituary framework [1] to inform an access provider of the remaining bid of a discarded packet. But this approach increases design complexity.

Instead, our design trades accuracy for simplicity. We let an access provider reduce the bidding cost of a sender $\sum b_s - \sum b'_s$ by a small amount to compensate for the average packet loss rate. If the average packet loss rate is ϵ , the access provider may reduce the sender's cost by $\epsilon * \sum b_s$. This provides incentives for a sender to keep its loss rate no higher than the average. As a packet loss wastes network resources, a well-behaved sender should always reduce its sending rate or increase its bid when the network is congested.

If a packet is discarded in one of a receiver's providers, the provider will be charged by its upper-level providers for the bidding cost of the packet, but the provider may not recover this cost from its customers. Normally, packet loss in the receiver's network will not be a significant problem, because a sender has the incentive to keep the loss rate low.

However, a problem may arise if a sender and a receiver collude to launch a DoS attack. A receiver may

authorize more traffic than it can receive, wasting upstream network resources without paying for it. In addition, a colluding sender may send traffic whose TTL expires before it reaches the receiver. We note that this problem is not caused by enabling packet auction: malicious hosts can already launch this type of attack in today's Internet. Although auction makes it worse, as the colluders may send TTL-expired packets with high bids, an solution that can limit this attack in today's Internet can also limit this attack in a network that supports packet auction.

3.6 Granularity of A Bid

The unit of bid in a packet header could be payment per packet, or payment per byte. Our design chooses the latter, because it simplifies bid comparison. A router compares bids in packet headers without considering the packet size, but when it charges a packet, it charges the market clearing price multiplied by the packet size. A granularity of payment per byte allows a small number of bids to represent a wide range of values as the packet size varies. For instance, if one unit of bid corresponds to one nano-dollar, and eight bits are used to represent a bid, a packet's value may range from 40 to $3.84 * 10^5$ nano-dollars, assuming 40 to 1500-byte packet sizes.

4. FEASIBILITY ANALYSIS

We analyze the feasibility of the design by examining its overhead. The design introduces memory, processing, and packet header overhead. The memory overhead is the per-customer accounting state, which we think is negligible. The processing overhead comes from 1) the queuing scheme to support an auction, 2) the hash computation to secure a sender's exit bid, and 3) capabilities. As we only use two weighted queues: a fast lane and a slow lane, we think various queuing operations can be done efficiently. Our openssl speed test shows that a hash computation derived from the block cipher AES takes 165 nano-seconds on a PC with an AMD Opteron 285 2.6GHz CPU. As each packet only requires one such computation, this CPU can perform the hash computations for six million packets every second. According to [19], the majority of capability packets have little processing overhead after capabilities are cached. If we assume CPU is the bottleneck resource, we think a prototype implementation of our design can support a few Gbps throughput, and a hardware optimized implementation may achieve an even higher throughput.

If we use an 8-bit bid, a 16-bit nonce, and a 32-bit hash value, we expect an eight-byte overhead for a sender's bid header and a nine-byte overhead for a receiver's bid header (eight bytes capabilities [19] plus one byte bid). Returning exit bids adds another seven-byte overhead, but multiple exit bids can be sent in one

packet. This header overhead is non-trivial given an Ethernet MTU of 1500 bytes. If the jumbo frame size of Gigabit Ethernet (9000 bytes) becomes more popular in the future, the header overhead is less a problem. Furthermore, if a user does not wish to receive better than best effort service, he can send packets without a bid header. This further reduces the header overhead. It is our future work to investigate how to reduce this header overhead.

5. DISCUSSION

This section discusses miscellaneous issues.

5.1 Billing a Flat-Fee Subscriber

Bidding costs may cause a customer's monthly charge to vary. For large customers, it is unlikely a problem because they are already charged usage-based fees. However, it has been shown [16] that individual subscribers prefer a flat-fee, even if the fee is higher than usage-based fees. We discuss how an access provider may support packet auctions with a flat-fee based billing scheme.

An access provider may allocate a subscriber an amount of "free" bidding tokens to avoid unpredictable usage-based charges, similar to the peak-time minutes in cellular phone plans. Each token corresponds to one unit of bid. A higher monthly fee is allocated more monthly tokens. If a user's traffic incurs a bidding cost, the cost is taken from his token bank. If a user uses up his tokens, it is a signal that he has consumed his share of bandwidth during network congestion. The user may purchase additional tokens to receive better than the best effort service, or send traffic without a bid header to receive the best effort service.

5.2 End Host Algorithms

Host software needs to be modified to take advantage of this auction framework. We imagine that each application may have a default bid value and a user can overwrite it according to his preferences and perceived performance. A software agent will assist a user to translate the amount he is willing to pay to a bid in a packet header. For instance, if a user is willing to pay 1 cent per minute for international VoIP, the software agent may translate this amount to a per packet bid by dividing the encoding rate of VoIP.

If a user's bid is low and his traffic is demoted to the slow lane, an inelastic application should use self-administered admission control to send later, or reduce its encoding rate but increase the per-packet bid to meet a user's budget. An elastic application may use TCP-like congestion control algorithm to adapt its sending rate to the available bandwidth.

5.3 Dishonest ISPs

Our design assumes that ISPs will honestly charge the Vickrey price. Dishonest ISPs may overcharge a packet. Presently, our design relies on market competition to discipline ISPs. If a dishonest ISP overcharges, a user may notice that his network cost is high, and switch to a lower cost provider. It is within our future work to design mechanisms for a user to detect misbehaving ISPs and switch providers at a finer time scale such as at the packet level.

6. CONCLUSION

This paper argues that packet auction may be a better alternative for ISPs to address the bandwidth scarcity problem than application blocking or discrimination. Users place bids in their packets. ISPs allocate bandwidth to the most valuable packets, and charge them the Vickrey auction price. We outline a technical design that addresses a number of challenges arising from supporting this auction. A preliminary analysis suggests that the design has reasonable overhead. It is our future work to implement and evaluate this design, and study how end systems may place bids in packets.

7. REFERENCES

- [1] K. Argyraki, P. Maniatis, D. Cheriton, and S. Shenker. Providing packet obituaries. In *ACM HotNets-III*, 2004.
- [2] R. Beverly, S. Bauer, and A. Berger. The Internet's not a big truck: Toward quantifying network neutrality. In *Proc. of PAM Workshop*, Apr. 2007.
- [3] B. Briscoe. Re-ECN: Adding accountability for causing congestion to TCP/IP. IETF draft, *draft-briscoe-tsvwg-re-ecn-tcp-04*, 2007.
- [4] BusinessWeek. Online extra: At SBC, it's all about "scale and scope", Nov. 7, 2005.
- [5] B. Charny. Vonage says its calls are still being blocked. CNET News.com, Mar. 21, 2005.
- [6] D. Clark. A model for cost allocation and pricing in the Internet. In *MIT Workshop on Internet Economics*, 1995.
- [7] D. Clark, J. Wroclawski, K. Sollins, and R. Braden. Tussle in Cyberspace: defining tomorrow's Internet. In *ACM SIGCOMM*, Aug. 2002.
- [8] C. Estan, A. Akella, and S. Banerjee. Achieving good end-to-end service using bill-pay. In *ACM HotNets-V*, 2006.
- [9] R. Gibbens and F. P. Kelly. Resource pricing and the evolution of congestion control. *Automatica*, 1999.
- [10] J. Krim. Executive wants to charge for web speed. Washington Post, Dec. 1, 2005.
- [11] V. Krishna. *Auction Theory*. Academic Press, 2002.
- [12] J. K. MacKie-Mason and H. R. Varian. Pricing the Internet. In *Public Access to the Internet*, May 1993.
- [13] M. Reardon. AT&T chief, FCC chair clarify on net neutrality. CNET News.com, Mar. 21, 2006.
- [14] N. Semret. *Market Mechanisms for Network Resource Sharing*. PhD thesis, Columbia University, 1999.
- [15] S. Shenker, D. D. Clark, D. Estrin, and S. Herzog. Pricing in computer networks: Reshaping the research agenda. *ACM CCR*, 1996.
- [16] H. R. Varian. The demand for bandwidth: Evidence from the INDEX project. <http://www.ischool.berkeley.edu/~hal/Papers/brookings/brookings.html>, Feb. 2002.
- [17] Network Neutrality. http://en.wikipedia.org/wiki/Net_neutrality.
- [18] X. Yang, G. Tsudik, and X. Liu. A technical approach to net neutrality. In *ACM HotNets-V*, Nov. 2006.
- [19] X. Yang, D. Wetherall, and T. Anderson. A DoS-limiting network architecture. In *ACM SIGCOMM*, Aug. 2005.